

# How the CBAM Corridor Cost Model Works

A walkthrough of the code, for the team

Samir Abuisneneh (Student 3), MSc Data Science for Business

7 August 2026

## Contents

|                                     |   |
|-------------------------------------|---|
| 1. What the program is              | 1 |
| 2. The shape of the code            | 1 |
| 3. What happens when you run it     | 2 |
| 4. One number, start to finish      | 3 |
| 5. How the code avoids lying to you | 3 |
| 6. Running it                       | 4 |
| 7. What it deliberately does not do | 4 |

Written 7 August 2026 by Samir (Student 3), for anyone on the team who wants to understand what the Python actually does without reading the Python.

This is a companion to `model_overview_for_team.md`, not a replacement for it. That document explains the project: who owns which input, what is still a placeholder, what the headline results are. This one explains the machine: what happens between pressing run and getting a number, and how much of that number you are entitled to trust.

---

## 1. What the program is

It is a calculator, not a simulation and not a forecast. You give it a corridor, a product, a production pathway, a year and a carbon price scenario. It gives you back what carbon regulation costs to move one tonne of that product along that corridor in that year.

Nothing in it is random. Run it twice with the same settings and you get the same number to the last decimal place, which is why the whole thing can be checked by tests rather than by eyeballing charts.

The two corridors are fixed, because the study is about them specifically:

| Corridor             | Physical route              | Rules that bite                           |
|----------------------|-----------------------------|-------------------------------------------|
| Halifax to Hamburg   | Canada to Germany           | EU CBAM, EU ETS Maritime, FuelEU Maritime |
| Ningbo to Felixstowe | China to the United Kingdom | UK CBAM from 2027, UK ETS Maritime        |

CBAM is the Carbon Border Adjustment Mechanism, a charge levied at the border on the carbon emitted making an imported good. ETS is an Emissions Trading System, which charges a ship for the carbon it emits on the voyage itself. FuelEU Maritime is a separate European penalty on the greenhouse gas intensity of the fuel a ship burns, and it applies to the European corridor only, because Felixstowe sits outside European jurisdiction.

## 2. The shape of the code

Roughly 9,700 lines of Python, and the split between the folders is the important part.

|                         |                                                       |
|-------------------------|-------------------------------------------------------|
| cbam_model/             |                                                       |
| config/                 |                                                       |
| regulatory_constants.py | every legal figure, each with its source in a comment |
| vessel_logistics.py     | Gayu's vessel and voyage arithmetic                   |
| scenarios.py            | the scenario grid and its display labels              |
| unresolved.py           | the sentinel that refuses to guess (see section 5)    |
| model/                  |                                                       |
| cbam.py                 | the border charge, EU and UK                          |
| ets_maritime.py         | the voyage charge, EU and UK                          |
| fuel_eu.py              | the fuel intensity penalty                            |
| total_cost.py           | joins the per-voyage and per-tonne halves             |
| switching.py            | corridor lock-in under long contracts                 |
| validation/             |                                                       |
| unit_checks.py          | rejects malformed input before it reaches the model   |
| gayu_reproduction.py    | pins Gayu's published figures as expected values      |
| reference_case.py       | checks the model against a published paper            |
| analysis/               |                                                       |
| outputs.py              | the results tables and charts                         |
| sensitivity.py          | what actually drives the answer                       |
| data_io.py              | loads and documents the three input tables            |
| runner.py               | runs the whole scenario grid                          |

Two rules hold across all of it. Every regulatory number lives in `regulatory_constants.py` and nowhere else, so there is one place to check a figure against the legislation. And no function invents a value it was not given.

### 3. What happens when you run it

Five steps, in this order.

**Load and check the inputs.** Three comma-separated value files: Riya's embedded emissions, Gayu's voyage logistics, and the commercial costs. Before any of it reaches a formula, `unit_checks.py` tests it against the agreed contract. It will reject a distance in kilometres presented as nautical miles, an emissions column in kilograms presented as tonnes, a corridor label with a typo in it, and a fuel total that does not equal distance times burn rate. The point is that a units error produces a plausible looking answer that is wrong by a factor of a thousand, and a plausible wrong answer is far more dangerous than a crash.

**Price the voyage.** Gayu's figures give fuel burned, which gives carbon dioxide emitted, which the European and British emissions trading systems charge for. The two schemes are not symmetrical and that asymmetry is the study's central finding. Europe charges half of an international voyage's emissions. Britain charges none of the ocean crossing and all of the time sitting in port. So for Ningbo to Felixstowe the entire sea passage is free and only the Felixstowe berth call costs anything.

**Price the border.** Embedded emissions times a phase-in factor times the carbon price, less any carbon price already paid in the producing country. Canada prices industrial carbon, so Canadian cargo arrives with a deduction. China does not yet price hydrogen or ammonia production, so Chinese cargo arrives with none.

**Join the two.** Here is the awkward part, and it is worth understanding because it is where the two halves of the group's work meet. Gayu's costs are per voyage. The border charge is per tonne of product. Converting between them needs to know how many tonnes a voyage carries, which her original notebooks never stated because they never needed to. Her cargo capacity notebook of 25 July supplied it: an 84,000 cubic metre carrier filled to the 98% regulatory limit holds 56,142 tonnes of ammonia or 5,828 tonnes of liquid hydrogen. That ratio of roughly nine to one is why hydrogen absorbs so much more shipping cost per tonne than ammonia does.

**Write the outputs.** Twenty-eight files land in `cbam_model/outputs/`, covering the cost tables, the corridor comparison, the sensitivity ranking and the charts.

## 4. One number, start to finish

This is the clearest way to see the whole machine. Take ammonia, Halifax to Hamburg, the regulatory default pathway, in 2030, at the medium carbon price. The model returns **54.67 euros per tonne**. Here is every step of where that comes from.

**The voyage.** A very large gas carrier covers 2,962 nautical miles at 14.8 knots, which is 8.3 days at sea. At that speed it burns 342.8 tonnes of fuel, which emits 1,097.0 tonnes of carbon dioxide equivalent. Carbon dioxide equivalent, written CO<sub>2</sub>e, bundles methane and nitrous oxide in with the carbon dioxide on a common warming scale, and both schemes have charged all three since 2026.

**The European emissions trading charge.** Only half of an international voyage counts.

1,097.0 tonnes × 0.50 coverage × 126.00 euros = 69,111.00 euros

**The fuel intensity penalty.** The ship burns conventional very low sulphur fuel oil at 90.8 grams of CO<sub>2</sub>e per megajoule. The 2030 target is 85.6904, so the ship is over by 5.1096 grams for every one of the 14,054,800 megajoules it consumed. That deficit gets converted into tonnes of fuel oil equivalent and charged at 2,400 euros each, which comes to **46,297.03 euros**.

Voyage total: **115,408.03 euros**.

**The border charge, per tonne.** Ammonia's regulatory default is 1.98 tonnes of CO<sub>2</sub>e per tonne of product. Because it is a regulatory default rather than a verified figure, a penalty mark-up applies, and for ammonia that mark-up is 1%. In 2030 the European scheme charges 48.5% of embedded emissions, with the rest still shielded by free allocation. Canada's industrial carbon price for 2030 is worth 71.75 euros per tonne and is deducted from the 126.00 euro certificate price.

1.98 × 1.01 mark-up = 1.9998 tonnes CO<sub>2</sub>e  
1.9998 × 0.485 = 0.9699 tonnes chargeable  
0.9699 × (126.00 – 71.75) = 52.62 euros per tonne

**The join.** Spread the voyage cost across the cargo.

115,408.03 euros ÷ 56,142 tonnes = 2.06 euros per tonne  
52.62 + 2.06 = 54.67 euros per tonne

Notice the proportions, because they carry a finding. By 2030 the border charge is roughly twenty-six times the shipping charge. The maritime side dominates in 2026 and is close to irrelevant by 2030, which means any conclusion drawn from a single year is really a conclusion about that year's position in the phase-in schedule.

That 1% mark-up is worth a second look. It was 30% in this model until 7 August, when checking the Commission's published default values revealed that fertiliser goods carry a flat 1% while everything else ramps to 30%. Ammonia is a fertiliser good for these purposes and hydrogen is not. Ammonia figures produced before that date are overstated, so regenerate rather than reuse them.

## 5. How the code avoids lying to you

This is the part that is unusual, and it exists because earlier drafts of this project got four separate regulatory facts wrong by assuming rather than checking.

**It refuses to guess.** A value nobody has sourced yet is not set to zero or to a sensible looking placeholder. It is set to an Unresolved object that raises an error on any arithmetic, any comparison, even any attempt to test whether it is true. A missing figure therefore stops the calculation at the exact line that needed it, with a message saying what to go and look up. Full delivered cost is blocked the same way: conversion and freight have no source, so asking for a delivered cost raises rather than returning a number built on invention.

**It reproduces Gayu's notebooks exactly.** Thirty-four of her published figures are written into the test suite as expected values. If anyone changes the fuel carbon factor or the engine load assumption, those tests fail and name the quantity that moved, rather than the model drifting quietly away from the maritime work it is built on.

**It refuses to call a coin flip.** Where the model compares a cost against a threshold, the verdict has three states rather than two. Anything within 10% of the line reads marginal instead of pass or fail. That band exists for a concrete reason: Chinese green ammonia comes out 1% below the carbon price, on a figure

assembled from two unrelated papers. Reported as a yes or no that is a confident yes. Reported honestly it is a coin flip.

**It keeps currencies apart.** European costs are in euros, British costs in pounds, and nothing converts silently. Where a chart does need one axis, the column says `gbp_equivalent` and the exchange rate is a single fixed reference rate from 23 July 2026, held constant across the horizon. That is a real limitation and it bites hardest wherever the two corridors sit close together.

**It labels what is not law.** Two of the three British carbon price paths are scenarios rather than legislation, and the proposed extension of the British scheme to international voyages is a consultation that never became law. Every one of these carries its status in its own label, and a test fails if a new scenario is ever added without one.

## 6. Running it

```
python3 -m venv .venv
.venv/bin/pip install -r requirements.txt -r requirements-dev.txt

.venv/bin/jupyter lab run_model.ipynb    # the whole thing, end to end
.venv/bin/python -m pytest -q             # the test suite
.venv/bin/streamlit run dashboard.py      # point and click version
```

The dashboard is the one to reach for if you do not want to read code. Pick a corridor, product, pathway, year and price scenario from the sidebar and it recalculates live. It calls the same tested functions the notebook does, so it cannot drift from the model, and anything policy-uncertain is labelled as such on screen rather than presented as a forecast.

## 7. What it deliberately does not do

Worth knowing before quoting anything from it.

It does not produce a full delivered cost. Production cost is real, from Riya's literature review. Conversion and freight are not sourced and have no owner, so they are a declared boundary rather than a pending input. What the model reports is the carbon regulation cost, which is a narrower and more defensible claim.

It does not optimise. There is no solver and nothing is being searched for. Where it picks a cheapest pathway it is ranking a handful of routes that exist in the literature. Calling it an optimisation model in the write-up would overclaim.

It does not forecast prices. The carbon price scenarios are a bracket, not a distribution, and no probability is attached to any of them.

It does not model a real liquid hydrogen ship, because no commercial fleet exists. The hydrogen figures apply an ammonia carrier's geometry to hydrogen deliberately, to isolate what cargo density alone does to cost per tonne. That is a counterfactual and should be labelled as one in the results.

---

*Questions to Samir. Every figure quoted here regenerates from the code, and the file to check any regulatory number against its source is `cbam_model/config/regulatory_constants.py`.*